

Probabilistic Logic Programming

Simon Frost

Overview

RDFLib.jl includes a ProbLog module for probabilistic logic programming with exact inference. ProbLog extends Prolog-style logic programming by allowing facts to have associated probabilities. The implementation uses BDD-based (Binary Decision Diagram) weighted model counting — the same algorithm as [ProbLog2](#) — to compute exact probabilities.

```
using RDFLib
```

Probabilistic Facts

In ProbLog, a fact can be annotated with a probability. The notation `0.5::heads.` means “heads is true with probability 0.5”.

```
result = problog_query("""
    0.5::heads.
    query(heads).
""")
println("P(heads) = $(result["heads"])")
```

P(heads) = 0.5

Coin Flip Example

With two independent coins, we can compute the probability of at least one showing heads:

```

result = problog_query("""
    0.5::heads1.
    0.6::heads2.
    someHeads :- heads1.
    someHeads :- heads2.
    query(someHeads).
""")
# P(someHeads) = 1 - P(¬heads1) × P(¬heads2) = 1 - 0.5 × 0.4 = 0.8
println("P(someHeads) = $(result["someHeads"])")

```

$P(\text{someHeads}) = 0.8$

Probabilistic Graphs

A classic ProbLog example: computing reachability in a graph where edges exist probabilistically.

```

result = problog_query("""
    0.9::edge(a,b).
    0.8::edge(b,c).
    0.7::edge(a,c).

    path(X,Y) :- edge(X,Y).
    path(X,Y) :- edge(X,Z), path(Z,Y).

    query(path(a,c)).
""")
# Two paths: direct (0.7) and via b (0.9 × 0.8 = 0.72)
# P = 1 - (1-0.7)(1-0.72) = 1 - 0.3 × 0.28 = 0.916
println("P(path(a,c)) = $(round(result["path(a,c)"], digits=4))")

```

$P(\text{path}(a,c)) = 0.916$

Non-ground Queries

Queries can contain variables — they are automatically expanded to all matching ground atoms:

```

result = problog_query("""
    0.9::edge(a,b).
    0.8::edge(b,c).
    0.5::edge(c,d).

    path(X,Y) :- edge(X,Y).
    path(X,Y) :- edge(X,Z), path(Z,Y).

    query(path(a,X)).
""")
for (q, p) in sort(collect(result))
    println("  P($q) = $(round(p, digits=4))")
end

```

```

P(path(a,b)) = 0.9
P(path(a,c)) = 0.72
P(path(a,d)) = 0.36

```

Evidence and Conditioning

Evidence allows you to condition on observed facts, computing posterior probabilities:

```

result = problog_query("""
    0.3::rain.
    0.5::sprinkler.
    0.9::wet :- rain.
    0.8::wet :- sprinkler.

    evidence(wet, true).
    query(rain).
    query(sprinkler).
""")
println("P(rain | wet) = $(round(result["rain"], digits=4))")
println("P(sprinkler | wet) = $(round(result["sprinkler"], digits=4))")

```

```

P(rain | wet) = 0.5018
P(sprinkler | wet) = 0.7598

```

Negative evidence works too:

```

result = problog_query("""
    0.3::rain.
    0.5::sprinkler.
    0.9::wet :- rain.
    0.8::wet :- sprinkler.

    evidence(wet, false).
    query(rain).
    query(sprinkler).
""")
println("P(rain | ¬wet) = $(round(result["rain"], digits=4))")
println("P(sprinkler | ¬wet) = $(round(result["sprinkler"], digits=4))")

```

$P(\text{rain} \mid \neg \text{wet}) = 0.0411$

$P(\text{sprinkler} \mid \neg \text{wet}) = 0.1667$

Negation-as-Failure

The $\backslash+$ operator represents negation-as-failure:

```

result = problog_query("""
    0.4::p.
    q :- \\\+p.
    query(q).
""")
# q is true when p is false: P(q) = 1 - P(p) = 0.6
println("P(q) = $(result["q"])")

```

$P(q) = 0.6$

Bayesian Network Example

A classic alarm network modeled in ProbLog:

```

result = problog_query("""
    0.1::burglary.
    0.2::earthquake.

    0.9::alarm :- burglary, earthquake.

```

```

0.8::alarm :- burglary, \\+earthquake.
0.1::alarm :- \\+burglary, earthquake.

0.8::calls(mary) :- alarm.
0.1::calls(mary) :- \\+alarm.
0.7::calls(john) :- alarm.
0.05::calls(john) :- \\+alarm.

evidence(calls(mary), true).
evidence(calls(john), true).
query(burglary).
query(earthquake).
"""
println("P(burglary | both call) = $(round(result["burglary"], digits=4))")
println("P(earthquake | both call) = $(round(result["earthquake"], digits=4))")

```

P(burglary | both call) = 0.7605
 P(earthquake | both call) = 0.3468

Lower-level API

For more control, use `parse_problog` and `problog_infer` separately:

```

prog = parse_problog("""
0.5::coin(1).
0.5::coin(2).
win :- coin(1), coin(2).
query(win).
""")

println("Clauses: ", length(prog.clauses))
println("Queries: ", length(prog.queries))

results = problog_infer(prog)
for (atom, prob) in results
  println("P($(atom.predicate)) = $prob")
end

```

Clauses: 3
 Queries: 1

```
P(coin) = 0.5  
P(coin) = 0.5  
P(win) = 0.25
```

What's Next?

- The ProbLog module matches Python ProbLog's exact inference results
- It uses BDD-based weighted model counting for precise probabilities
- For non-probabilistic logic, see the [Datalog vignette](#)
- For N3 rule-based reasoning, see the [N3 Reasoning vignette](#)